UNIVERSITY OF PETROLEUM AND ENERGY STUDIES (UPES)

Containerization-and-DevOps

Documentation File

Submitted to:

Dr. Prateek Raj Gautam

Submitted by:

Siddharth Negi

SAP ID- 500121910

Containerized Student Records Application (Final Submission)

---

1. Project Overview

Field	Details
Project Name	student-records-app
Backend	Node.js (v20-alpine) — Express REST API
Database	PostgreSQL 16 (alpine)
Networking	Docker MacVLAN (student_macvlan)
Orchestration	Docker Compose
API IP Address	172.21.0.100:3000
DB IP Address	172.21.0.101:5432

---

2. Project Structure

```
student-records-app/
├── backend/
│   ├── Dockerfile
│   ├── index.js
│   ├── package.json
│   └── .dockerignore
├── database/
│   ├── Dockerfile
│   └── init.sql
├── docker-compose.yml
├── network_setup.sh
└── .env
```

---

3. Component Description

3.1 Backend (Node.js)

The backend is built using Express and provides REST APIs:

Endpoint	Description
GET /health	Health check
GET /students	Retrieve all records
POST /students	Insert student

3.2 Backend Dockerfile

- Multi-stage build used
- Non-root user added
- Uses node:20-alpine

backend > Dockerfile > ...

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .

FROM node:20-alpine
RUN adduser -D appuser
WORKDIR /app
COPY --from=builder /app .
USER appuser
EXPOSE 3000
CMD ["node", "index.js"]
```

3.3 Database (PostgreSQL)

- Custom image using postgres:16-alpine
- init.sql creates students table automatically

database > init.sql

```
CREATE TABLE IF NOT EXISTS students(
 id SERIAL PRIMARY KEY,
 name VARCHAR(100),
 roll_number VARCHAR(20),
 department VARCHAR(50),
 year INT
);
```

---

4. Docker Compose

- Defines backend + database services
- Uses macvlan network
- Assigns static IP

```yaml
docker-compose.yml > { } services
Run All Services
services:

  Run Service
  backend:
    build: ./backend
    container_name: studentapi
    restart: always

    environment:
      DB_HOST: 172.21.0.101
      DB_USER: admin
      DB_PASS: admin123
      DB_NAME: studentdb
      DB_PORT: 5432

    networks:
      student_macvlan:
        ipv4_address: 172.21.0.100

    depends_on:
      db:
        condition: service_healthy

    healthcheck:
      test: ["CMD", "wget", "-qO-", "http://localhost:3000/health"]
      interval: 20s
      timeout: 5s
      retries: 3

  Run Service
  db:
    build: ./database
    container_name: studentdb
```

---

5. Macvlan Network Setup

docker network create \

--driver macvlan \

--subnet=172.21.0.0/16 \

--gateway=172.21.0.1 \

-o parent=eth0 \

student_macvlan

```
siddharth@Siddharth:/mnt/c/Users/Siddharth/Desktop/containerized-webapp/backend$ docker network create \
--driver macvlan \
--subnet=172.21.0.0/16 \
--gateway=172.21.0.1 \
-o parent=eth0 \
student_macvlan
f28e2c77ff7d1b42c4935d671d778c6cec1e7d477a7eca30afcb225113b95c45
```

---

6. Build & Deployment

docker compose up --build -d

```
 => => exporting manifest sha256:16b4c7196192d2089d1fed23c310223462c785c7394760c9989a72fa1215a  0.0s
 => => exporting config sha256:9f1af13ba9b4678193e1a791ab05b46065eaa1217da3f64f0081bbd95aff7d2a  0.0s
 => => exporting attestation manifest sha256:e65e4457b268ad255ff62b5346f90322309fcf1cd9dba49a36  0.1s
 => => exporting manifest list sha256:26c63a12976a17125f140cdf0e393602dc0d2c66a590aeddc7ba020e9  0.0s
 => => naming to docker.io/library/containerized-webapp-backend:latest                          0.0s
 => => unpacking to docker.io/library/containerized-webapp-backend:latest                       0.2s
 => [backend] resolving provenance for metadata file                                            0.0s
[+] up 5/5
 ✔Image containerized-webapp-db              Built                                                  8.9ss
 ✔Image containerized-webapp-backend         Built                                                  8.9ss
 ✔Volume containerized-webapp_student_pgdata Created                                               0.0ss
 ✔Container studentdb                        Healthy                                                21.1s
 ✔Container studentapi                       Created                                                0.1ss
```

---

7. Verification & Testing

7.1 Health Check

docker exec -it studentapi sh

wget -qO- http://localhost:3000/health

Output:

{"status":"ok"}

```
/app $
/app $
/app $
/app $
/app $ wget -qO- http://localhost:3000/health
{"status":"ok"}/app $
```

7.2 Insert Student Record

wget -qO- \

--header="Content-Type: application/json" \

--post-data='{"name":"Siddharth","roll_number":"CS01","department":"CSE","year":3}' \

http://localhost:3000/students

Output:

{"id":1,"name":"Siddharth","roll_number":"CS01","department":"CSE","year":3}

```
/app $ wget -qO- \
> --header="Content-Type: application/json" \
> --post-data='{"name":"Siddharth","roll_number":"CS001","department":"CSE","year":3}' \
> http://localhost:3000/students
{"id":1,"name":"Siddharth","roll_number":"CS001","department":"CSE","year":3}/app $
```

7.3 Fetch Records

wget -qO- http://localhost:3000/students

Output:

[{"id":1,"name":"Siddharth","roll_number":"CS01","department":"CSE","year":3}]

```
/app $
/app $
/app $
/app $
/app $ wget -qO- http://localhost:3000/students
[{"id":1,"name":"Siddharth","roll_number":"CS001","department":"CSE","year":3}]/app $
```

---

8. Network Verification

docker network inspect student_macvlan

```
siddharth@Siddharth:/mnt/c/Users/Siddharth/Desktop/Containerized-Webapp/backend$ docker network inspect student_macvlan
[
    {
        "Name": "student_macvlan",
        "Id": "f28e2c77ff7d1b42c4935d671d778c6cec1e7d477a7eca30afcb225113b95c45",
        "Created": "2026-03-20T11:46:27.02783217Z",
        "Scope": "local",
        "Driver": "macvlan",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.21.0.0/16",
                    "Gateway": "172.21.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {
            "parent": "eth0"
        },
        "Labels": {},
        "Containers": {
            "0eae97a6a9e30204a3d230e1d4aa8c0f6ea022e4f5bf0fa2f4310b760685cb6a": {
                "Name": "studentapi",
                "EndpointID": "71802e442e0a679e1aa2c3729a911444b51c964a04d4c643ddba8e9242158100",
                "MacAddress": "1e:b4:32:15:bc:fe",
                "IPv4Address": "172.21.0.100/16",
                "IPv6Address": ""
            },
            "753469788c77e5b7ee0d03956d8589db841999ff47a5606f629cd45bbd0d9c4f": {
                "Name": "studentdb",
                "EndpointID": "8f88175f853adc3003dc5eb3cb89a415318d2a7ba7589d2855eb07c33055da32",
                "MacAddress": "0a:71:3b:8e:30:35",
                "IPv4Address": "172.21.0.101/16",
                "IPv6Address": ""
            }
        },
        "Status": {
            "IPAM": {
                "Subnets": {
                    "172.21.0.0/16": {
                        "IPsInUse": 5,
                        "DynamicIPsAvailable": 65531
                    }
                }
            }
        }
    }
]
```

---

9. Persistent Storage

- Volume: student_pgdata
- Data stored in /var/lib/postgresql/data

Test:

docker compose down

docker compose up -d

```
siddharth@Siddharth:/mnt/c/Users/Siddharth/Desktop/containerized-webapp/backend$ docker volume inspect containerized-webapp_student_pgdata
[
    {
        "CreatedAt": "2026-03-20T12:00:41Z",
        "Driver": "local",
        "Labels": {
            "com.docker.compose.config-hash": "20373231253f30829038f2c2d4de8e3161ccc9dae54675a15dbb1fa427ca6b5a",
            "com.docker.compose.project": "containerized-webapp",
            "com.docker.compose.version": "5.0.2",
            "com.docker.compose.volume": "student_pgdata"
        },
        "Mountpoint": "/var/lib/docker/volumes/containerized-webapp_student_pgdata/_data",
        "Name": "containerized-webapp_student_pgdata",
        "Options": null,
        "Scope": "local"
    }
]
```

---

10. Running Containers

docker ps

```
siddharth@Siddharth:/mnt/c/Users/Siddharth/Desktop/containerized-webapp/backend$ docker ps
CONTAINER ID   IMAGE                        COMMAND                  CREATED          STATUS                    PORTS     NAMES
0eae97a6a9e3   containerized-webapp-backend   "docker-entrypoint.s…"   26 minutes ago   Up 26 minutes (healthy)             studentapi
753469788c77   containerized-webapp-db        "docker-entrypoint.s…"   26 minutes ago   Up 26 minutes (healthy)             studentdb
siddharth@Siddharth:/mnt/c/Users/Siddharth/Desktop/containerized-webapp/backend$
```

---

11. Important Note

Due to macvlan networking, host cannot access containers directly.
Testing is performed inside container.

---

12. Conclusion

- Successfully containerized Node.js + PostgreSQL app
- Implemented macvlan networking with static IP
- Verified API endpoints
- Achieved persistent storage using Docker volume